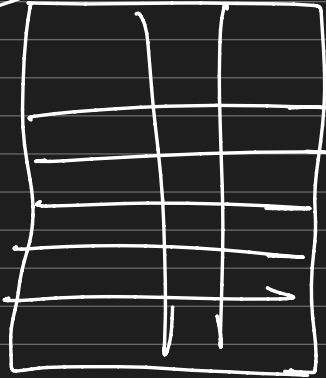


Overfitting / Underfitting

Ex: Math suggestion

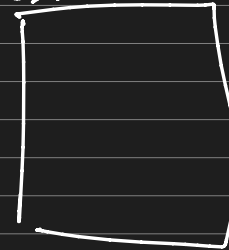


Test (suggestion)



Better
Better

Test (suggestion) based



Better
Bad

→ Generalized well

Better (Pattern)

Train → Better
Test → Better

Good fit

Bad (guess / Memorize)

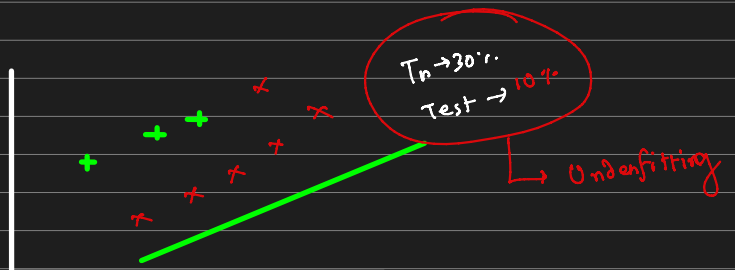
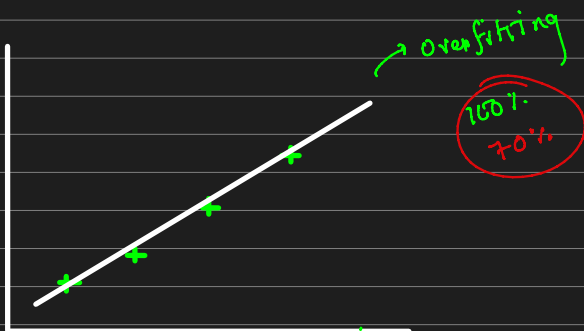
Train → Better
Test → Bad

Overfitting

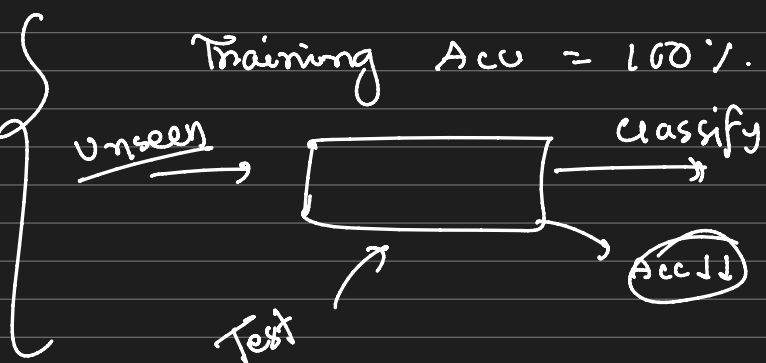
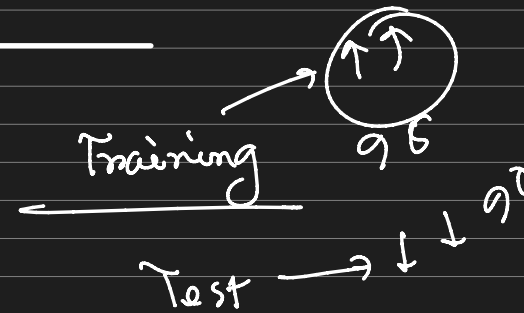
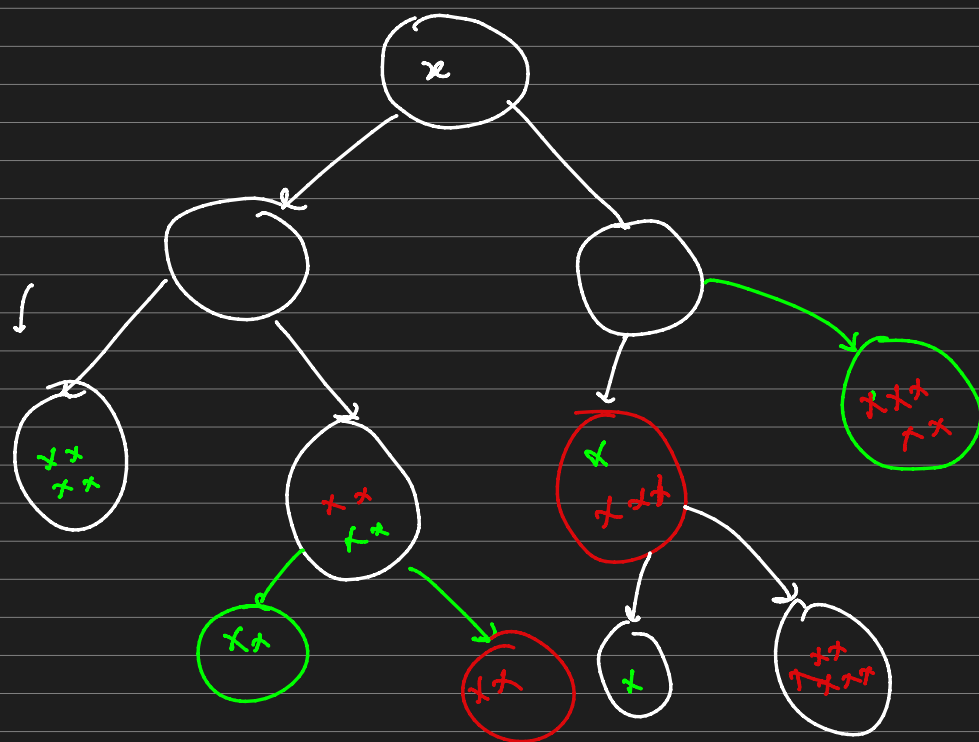
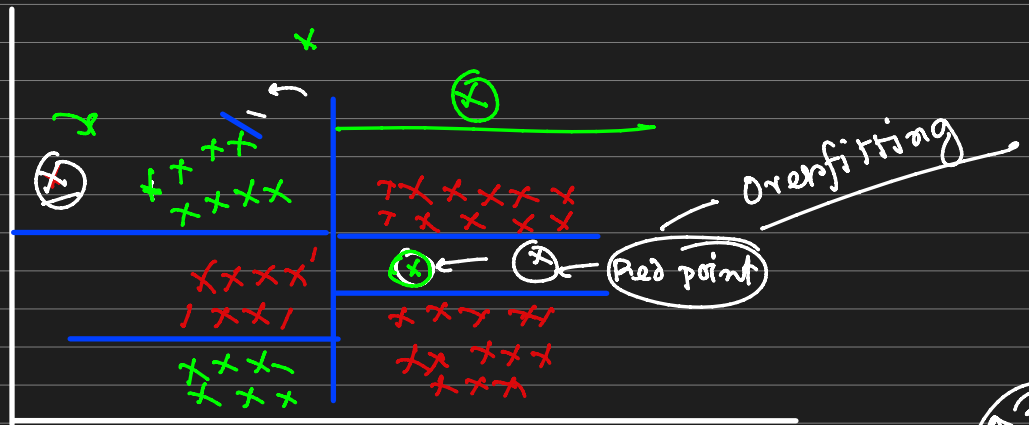
Shana Bad

Train → Bad
Test → Bad

→ Underfitting

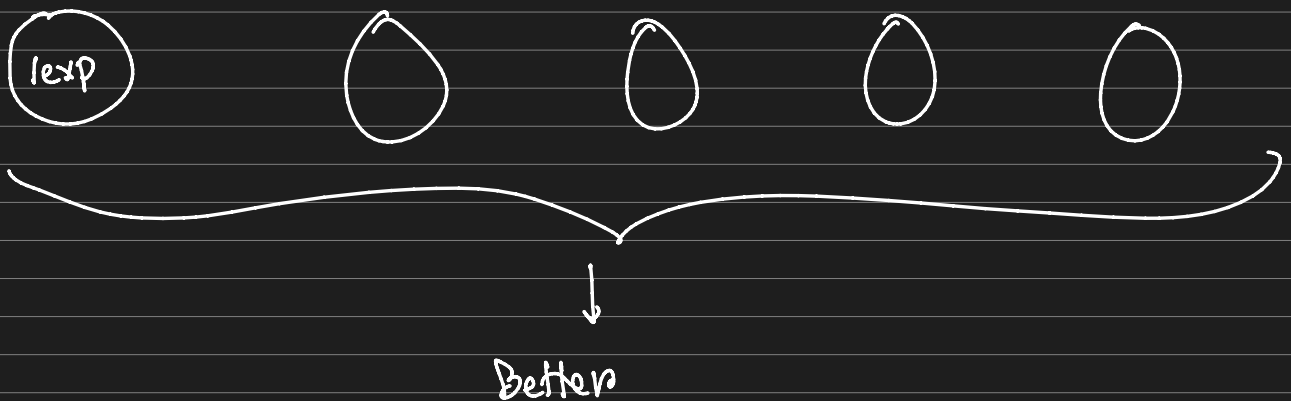
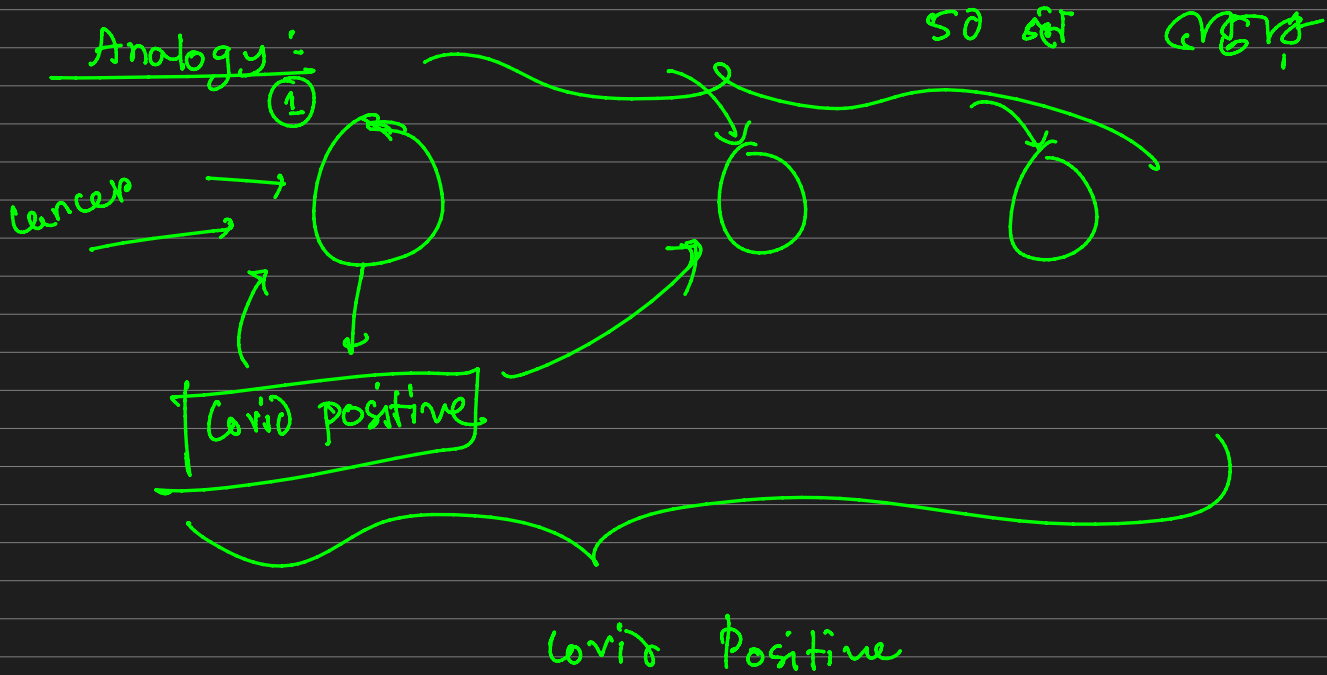
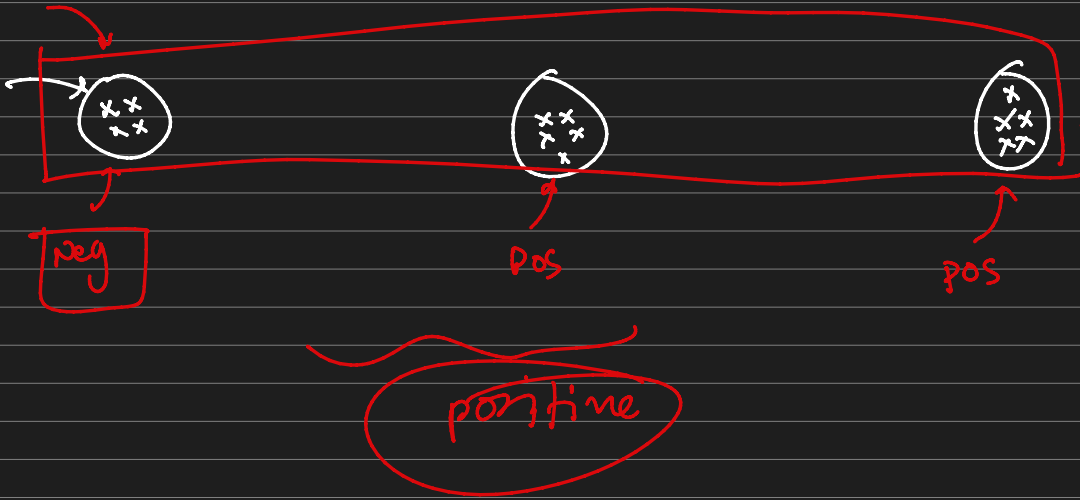


Over fitting on Decision Tree

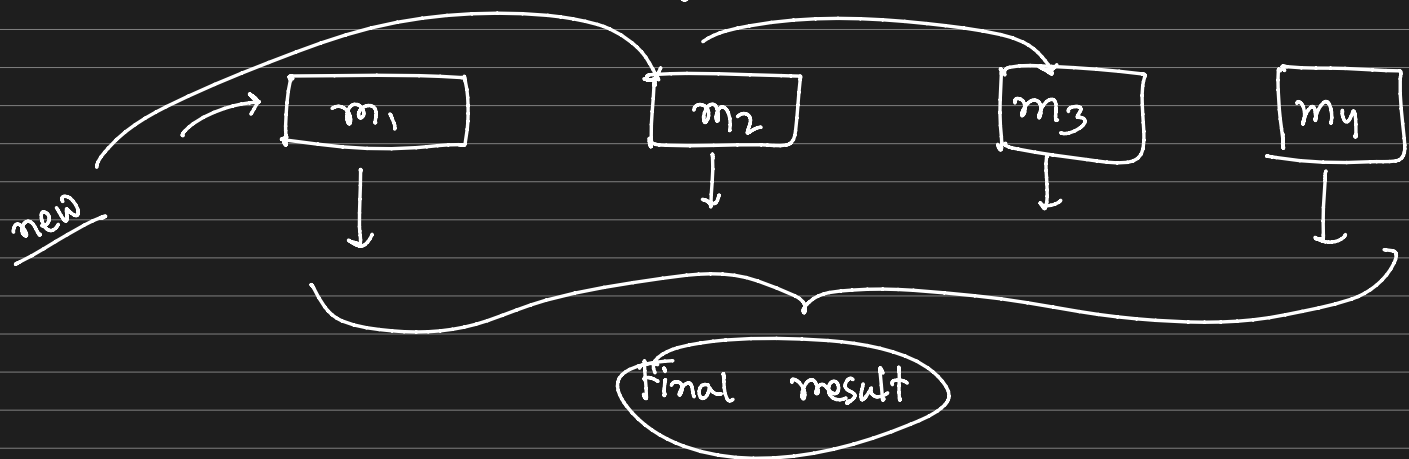


Wisdom of the crowd

The "wisdom of the crowd" is the concept that large, diverse, and independent groups of people often make better collective judgments, predictions, or problem-solving decisions than even individual experts. This phenomenon relies on aggregating individual errors, which naturally cancel each other out to reveal a highly accurate average. [Investopedia +2](#)



Problem → Regre / Classification



Ensemble learning

Random Forest

Grid Search

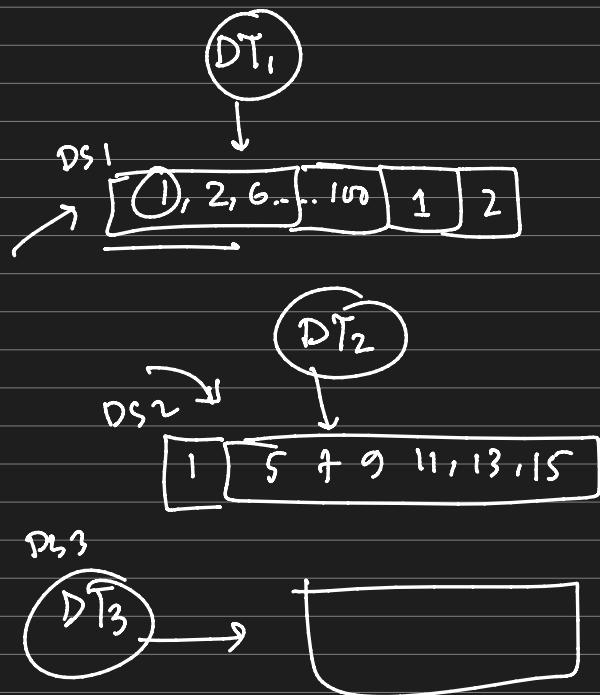
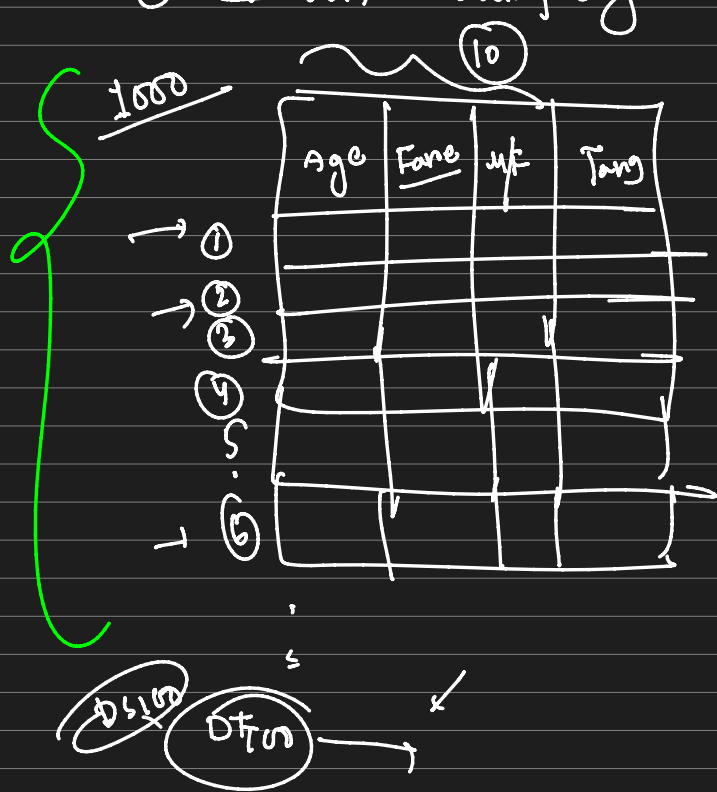
Multiple Decision Tree



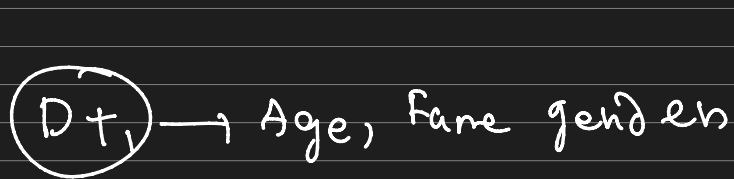
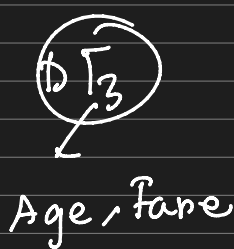
Random: Given dataset

- ① Random Sampling
- ② Random feature Sampling
- ③ Combined

i) Random Sampling

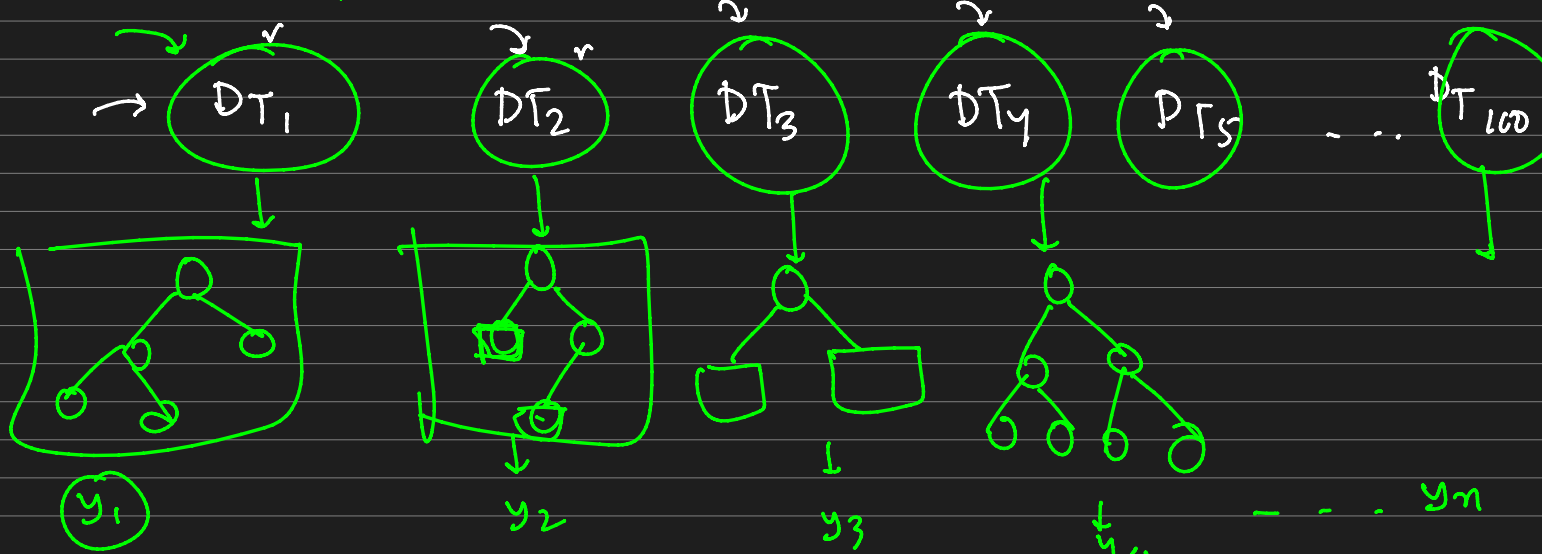


ii) Random feat / feat



iii)

Random



Prediction:

$[x_1 \ x_2 \ x_3] \rightarrow$

Aggregate

Bagging Algorithm

Classification:

$y_1 \ y_2 \ y_3 \ - \ - \ - \ y_n$

↓
Yes = 60 ✓ No = 40

→ Yes

Regression:

Base Model:

Decision Tree Regressor

Bagging } Bootstrapping, Aggregation

Summary:

① Random forest uses collection of decision tree to make predictions ✓

② These Decision Trees are trained on different data bootstrapped from the original dataset.

③ After the training, we get predictions from each Tree & aggregate them to output final prediction

④ Whole Process is called Bagging.

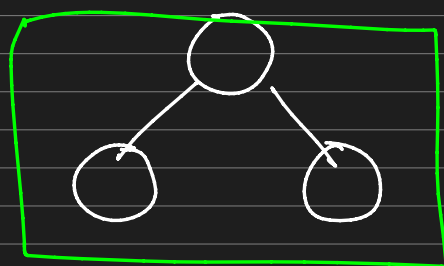
⑤ Combining multiple model for a single problem is called Ensemble Learning

Bias Variance Tradeoff

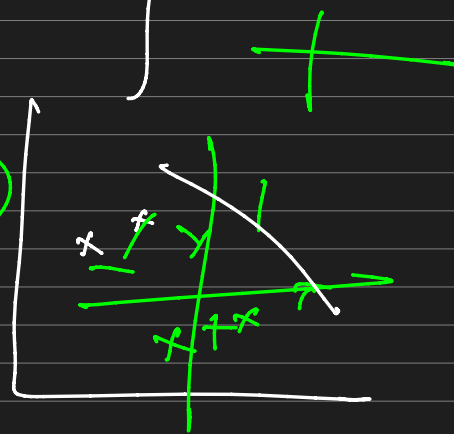
① Bias, Variance

① Model's complexity:

① $\begin{pmatrix} \overset{1}{-} & \overset{2}{x_1} & \overset{2}{-} & \overset{2}{-} \end{pmatrix} \rightarrow \text{Decision 1}$
 $\begin{pmatrix} - & \underline{x_2} & - & - \end{pmatrix} \rightarrow D_1$
 $\begin{pmatrix} - & - & \underline{x_2} & - \end{pmatrix} \rightarrow D_1$



simple

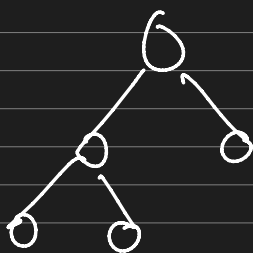


High Bias $\rightarrow$ High Simplicity

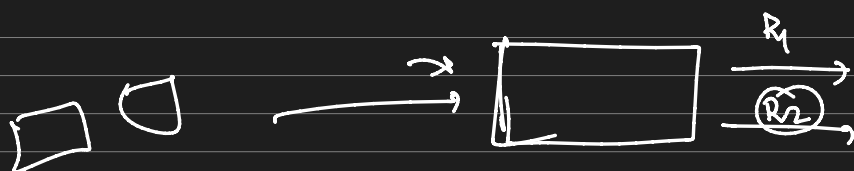
low Bias $\rightarrow$ low "

Simplicity $\propto \frac{1}{\text{comp}}$

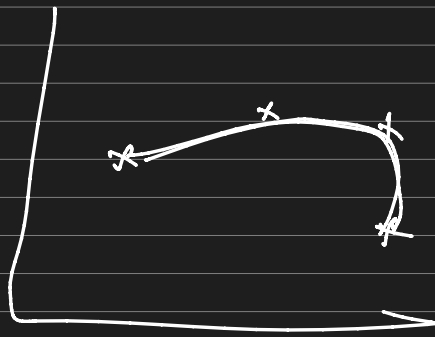
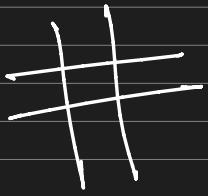
①①



①① Variance $\rightarrow$ Adaptability with training data



High variance $\rightarrow$ Sensitive $\uparrow$

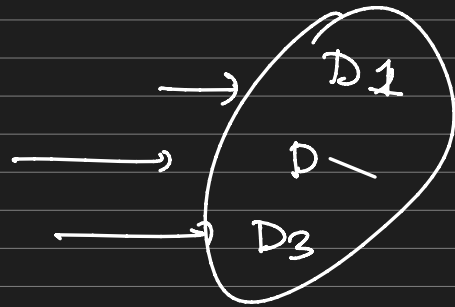
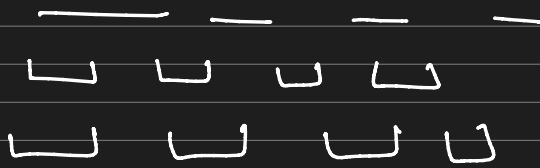


issue
overfitting

low Variance $\rightarrow$ less sensitive to data.

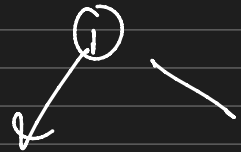
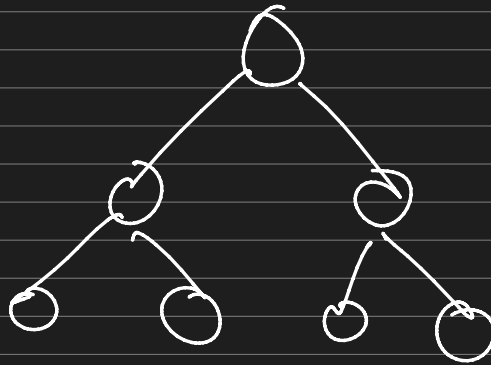
Ex:

①



Variance

$\rightarrow$ Model's sensitivity to data



High Variance $\rightarrow$ low stability.

Model $\rightarrow$ low Bias + low Variance

Variance $\downarrow$
Bias $\downarrow$

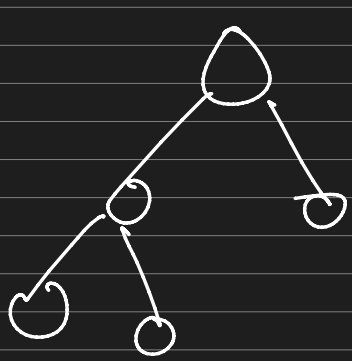
Bias $\uparrow$

Variance $\uparrow$

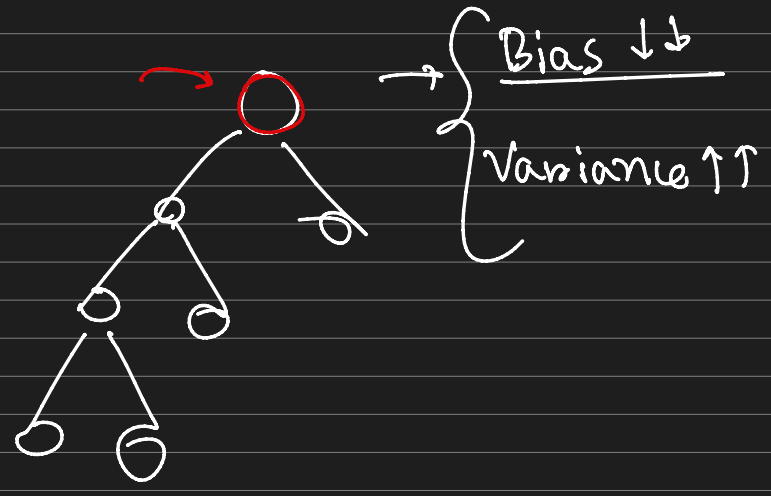
Bias $\rightarrow$ Variance $\rightarrow$ Generalized Mod

Balance

$\rightarrow$ Bias-Variance tradeoff



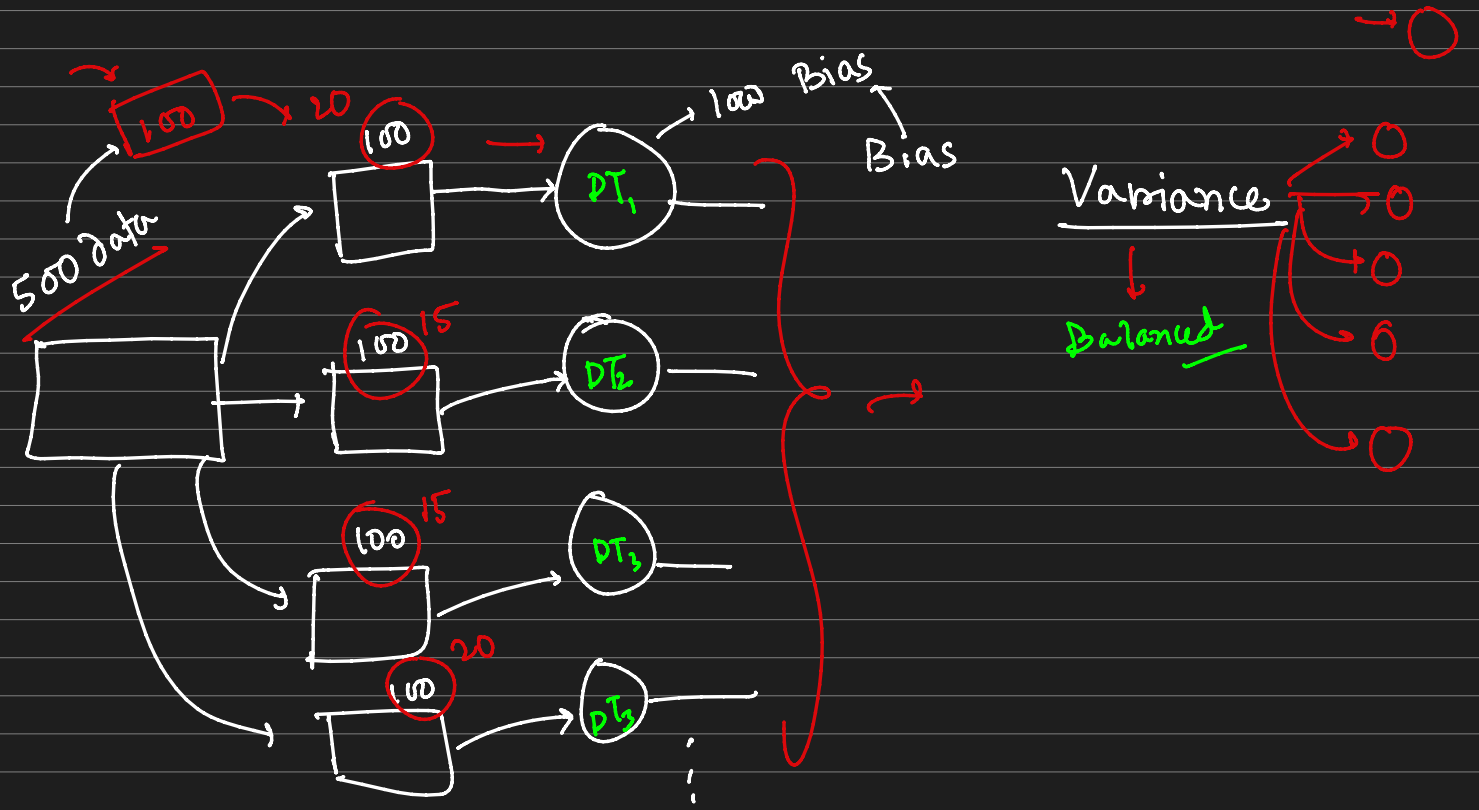
Bias ↑ Variance ↓



Why Random forest works better

DT = Low Bias High Variance

Random → low Bias low Variance ↓↓



Let's Code Random forest

Hyper Parameters

(i) n_estimators: How many DT to make
↳ (20 or 100) → tradeoff: Time

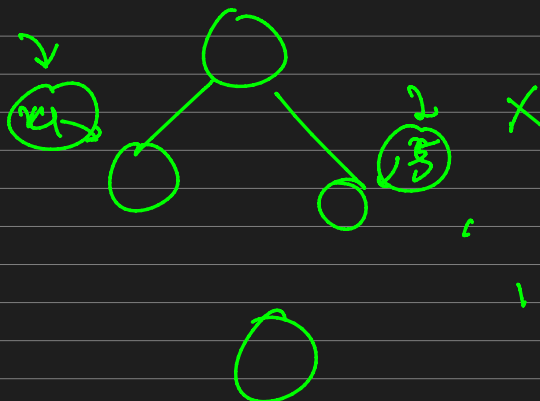
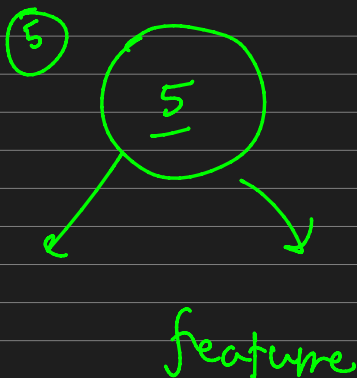
(ii) Criterion: Which information gain to use
↳ 'gini' →
↳ 'entropy' } Hyper parameters

(iii) max_depth: max depth to grow the trees

(iv) min_sample_split: minimum data points split reqd.

(v) min_sample_leaf: leaf node min reqd

(vi) max_feature: 'sqrt', 'log2'.



$$\sqrt{16} = 4$$

$$\text{sqrt}(4) = 2$$

1 2 3 4